



Universidade do Minho
Escola de Engenharia

Tecnologias de Sistemas Operativos

Trabalho Prático

Grupo 04

10/05/2026

Ano Letivo 2025/2026

Samuel Gibson da Cunha Figueiredo Lobato — A106907

Afonso Miguel Carvalho de Jesus Quartas — A106880

Lucas Rafael da Cunha Franco Robertson — A89467

Índice

1. Introdução
2. Arquitetura e Desenho da Solução
 - 2.1. Estruturas de Dados e Metadados
 - 2.2. Controlo de Concorrência
 - 2.3. Mecanismo de Deduplicação
 - 2.4. Operações de Leitura e Remoção
 - 2.5. Persistência
3. Ferramentas de Avaliação Experimental
 - 3.1. Syscounter
 - 3.2. Systracer
 - 3.3. Fio
 - 3.4. Pidstat
4. Discussão de Resultados
 - 4.1. Poupança de Espaço
 - 4.2. Desempenho e Sobrecarga
5. Conclusão

1. Introdução

O presente relatório descreve a implementação e avaliação de um sistema de ficheiros com suporte a deduplicação de dados, desenvolvido no âmbito da unidade curricular de Tecnologias de Sistemas Operativos da Universidade do Minho. O objetivo principal do trabalho consistiu em criar um sistema eficiente para arquivo de dados que permita poupar espaço em disco através da eliminação de blocos duplicados, mantendo a transparência para o utilizador final.

O sistema foi desenvolvido com base na plataforma FUSE (*Filesystem in Userspace*), estendendo uma implementação base *passthrough*. A solução foca-se num cenário de arquivo de dados com escrita em modo *append-only*, onde as operações de leitura e escrita são efetuadas com um tamanho mínimo e múltiplo de 4KB (4096 bytes), garantindo o alinhamento com os limites dos blocos. Desta forma, os ficheiros nunca são reescritos; apenas é possível adicionar conteúdo ao final de cada ficheiro.

A técnica de deduplicação implementada opera ao nível de blocos de tamanho fixo (4KB), utilizando a função de *hash* criptográfica SHA-512 da biblioteca OpenSSL para identificar univocamente cada bloco. A deduplicação é realizada de forma *inline*, ou seja, no caminho crítico da escrita, antes dos dados serem efetivamente persistidos no disco. Blocos duplicados entre diferentes ficheiros são eliminados, guardando-se apenas uma cópia física por bloco único.

Para além da funcionalidade de deduplicação, o sistema suporta a criação e listagem de diretórias, a criação, leitura e escrita de ficheiros, bem como a remoção de ficheiros e pastas com libertação efetiva de espaço. A avaliação experimental foi conduzida com recurso a ferramentas eBPF desenvolvidas especificamente para monitorizar chamadas ao sistema e rastrear operações em múltiplas camadas da pilha de armazenamento, juntamente com a utilização de outras ferramentas como o *fi* e o *pidstat*.

2. Arquitetura e Desenho da Solução

A arquitetura do sistema foi desenhada para suportar a deduplicação de blocos de 4KB de forma transparente ao utilizador. Para tal, foi necessário separar a representação lógica dos ficheiros (vista pelo utilizador) da sua representação física no disco. O sistema é composto por quatro módulos principais: o módulo FUSE (*passthrough.c*), o módulo de metadados (*metaInfo.c/metaInfo.h*), o módulo de persistência (*persistance.c/persistance.h*) e o módulo de locks de escrita e leitura (*ticket_rwlock.c/ticket_rwlock.h*).

Todas as estruturas de dados geridas pelo FUSE são guardadas em (*/backend/.metadata*), já os dados dos ficheiros são armazenados num único ficheiro de *backend* (*/backend/.sysdata*), onde cada bloco único ocupa uma posição contígua identificada pelo seu *offset*. Esta abordagem simplifica a gestão de armazenamento e permite que o sistema funcione sobre qualquer sistema de ficheiros subjacente.

2.1. Estruturas de Dados e Metadados

Para gerir a correspondência entre a visão lógica e física, foram implementadas estruturas de dados específicas, definidas no ficheiro *metaInfo.h*. A estrutura **filemeta** representa os

metadados de um ficheiro e contém o caminho do ficheiro (`id`), o tamanho real ocupado em disco após deduplicação, o tamanho lógico visível para o utilizador, uma lista ordenada de blocos que compõem o ficheiro, e um *mutex* personalizado, que irá ser explicado numa secção posterior, para garantir a consistência em acessos concorrentes.

A estrutura **blockmeta** representa os metadados de um bloco físico único. Armazena o *offset* do bloco no ficheiro de *backend*, a assinatura criptográfica SHA-512 que o identifica de forma única, o tamanho do bloco, e um contador de referências que indica quantos ficheiros partilham este bloco.

O contexto global do sistema de ficheiros mantém duas *hash tables* da biblioteca GLib cada uma com o seu próprio *lock*. A primeira, **fileIndex**, mapeia o caminho de um ficheiro para a sua estrutura `filemeta`. A segunda, **blockIndex**, mapeia a assinatura SHA-512 de um bloco para a sua estrutura `blockmeta`. Adicionalmente, mantemos uma lista de blocos vazios, também protegida com o seu próprio *lock*, é uma variável que regista o *offset* livre no final do ficheiro de dados dos blocos, este também protegido com *locks*.

2.2. Mecanismo de Controlo de Concorrência

O sistema foi desenhado para suportar múltiplos *threads* em simultâneo, uma vez que o FUSE pode invocar as operações do sistema de ficheiros de forma concorrente. Devido aos constrangimentos que o uso de *locks* implicam na performance do sistema, criamos um sistema personalizado de *locks* de leitura e escrita, gestão de fila de espera e permissão de elevação de permissões, passando dos *locks* de leitura para os de escrita quando necessário. No nosso sistema a ordem de entrada das secções críticas segue a seguinte ordem:

1. Índice de metadados dos ficheiros
2. Metadados dos ficheiros
3. Índice dos blocos (estes são adquiridos ordenadamente)
4. Metadados dos blocos

Garantindo assim consistência e a ausência de bloqueios durante a vida do sistema de ficheiros. Para garantir que a performance não é afetada, as operações pesadas mais pesadas são feitas fora das zonas críticas, não impedindo outras *threads* de entrar nas mesmas.

2.3. Mecanismo de Deduplicação

A deduplicação ocorre no caminho crítico das operações de escrita. Quando um pedido de escrita é recebido, o sistema divide os dados em blocos de 4KB. Para cada bloco, é calculada a sua assinatura utilizando a função `SHA512` da biblioteca OpenSSL, produzindo um *hash* de 512 bits. Esta assinatura serve como chave na tabela `blockIndex`.

Após o cálculo do *hash*, o sistema procura a assinatura na tabela `blockIndex`. Se o bloco já existir, o sistema não escreve os dados no disco. Em vez disso, incrementa o contador de referências do bloco existente e adiciona a referência à lista de blocos do ficheiro. Se o bloco for novo, o sistema aloca um novo *offset* no ficheiro de *backend*, escreve os dados no ficheiro de blocos, cria uma nova estrutura `blockmeta` com contador inicial a 1, e insere-a no `blockIndex`.

Este processo garante que apenas uma cópia de cada bloco único é armazenada fisicamente, independentemente de quantos ficheiros o contêm. Sempre que um *thread* aumenta o seu privilégio esta verifica se o bloco entretanto já não foi escrito na tabela

2.4. Operações de Leitura e Remoção

Na operação de leitura, o sistema consulta a lista de blocos do ficheiro e calcula quais os blocos físicos que correspondem ao *offset* lógico solicitado. A leitura é então traduzida em leituras físicas no ficheiro de *backend*. Para encontrar o bloco correspondente ao *offset* solicitado, é feita uma divisão entre o *offset* e o tamanho padrão dos blocos para assim encontrar em tempo $O(1)$ a posição do bloco solicitado na lista de blocos do ficheiro.

A remoção de ficheiros foi implementada para libertar espaço de forma eficiente. Quando um ficheiro é apagado, o sistema itera sobre a sua lista de blocos e decreenta o contador de referências de cada bloco. Se o contador de um bloco chegar a zero, significa que nenhum outro ficheiro o utiliza. Nesse caso, o *offset* do bloco é adicionado à lista de blocos livres para futura reutilização, e a entrada é removida do `blockIndex`, adicionalmente também apagamos o ficheiro através de `unlink`. A remoção de diretórias é suportada pela função `xmp_rmdir`, que delega a operação ao sistema de ficheiros subjacente.

2.5. Persistência

A persistência dos metadados é garantida pelas funções implementadas em `persistence.c`. O estado do sistema (tabelas `fileIndex` e `blockIndex`) pode ser serializado para um ficheiro binário (`/backend/.metadata`) através da função `save_metadata()` e posteriormente carregado no arranque através de `load_metadata()`. A serialização utiliza um formato binário compacto que guarda, para cada ficheiro, o seu caminho, tamanhos real e lógico, e a sequência de assinaturas SHA-512 dos blocos que o compõem. Para o índice global de blocos, são guardadas as assinaturas e os respectivos contadores de referência.

As funções auxiliares de I/O (`write_uint64`, `write_int64`, `write_bytes`, `write_string` e as correspondentes funções de leitura) implementam a serialização de tipos primitivos de forma portátil. O mecanismo de persistência utiliza um padrão de propagação de erros (`SaveCtx`) que interrompe a escrita caso ocorra qualquer falha durante a serialização.

3. Ferramentas de Avaliação Experimental

Para avaliar o impacto da deduplicação, foram utilizadas duas ferramentas fornecidas nas aulas práticas, baseadas em eBPF (*extended Berkeley Packet Filter*) que permitem monitorizar o comportamento do sistema em tempo real, sem introduzir perturbações significativas no desempenho medido. Para além destas também foram usados o `pidstat` e o `fio`.

3.1. Syscounter

A utilização do **Syscounter** permitiu contar o número de chamadas ao sistema e os bytes transferidos por processo. O programa instrumenta os *tracepoints* de entrada das chamadas

`openat`, `write`, `pwrite64`, `read`, `pread64` e `close`, filtrando apenas operações sobre ficheiros regulares.

3.2. Systracer

Já com o **Systracer**, esta ferramenta permitiu uma análise mais detalhada das operações em múltiplas camadas da pilha de armazenamento: Syscall, VFS, EXT4, FUSE e BIO (bloco). Para cada evento, regista o PID, o nome do comando, o identificador da chamada, a camada, o nome do ficheiro, o tamanho da operação e, no caso de escritas, um *hash* XXHash32 do conteúdo do *buffer*.

O cálculo do *hash* XXHash32 no *kernel* permite verificar se o conteúdo dos *buffers* de escrita se mantém consistente entre camadas, e identificar padrões de duplicação ao nível do *kernel*. A implementação do XXHash32 foi adaptada para ser compatível com o verificador BPF, limitando o tamanho máximo do *buffer* processado a 4096 bytes.

3.3. Fio

Utilizamos esta ferramenta pois isto permitiu-nos criar simulações de operações de escrita e leitura, simulando assim uma carga sintética aproximada da realidade. Esta ferramenta também foi escolhida pela sua possibilidade de medir a performance em MB/s das escritas e leituras do sistema.

3.4. Pidstat

Através desta ferramenta fomos possibilitados a medir a carga que a nossa implementação introduz na escrita e leitura em comparação com o sistema base.

4. Discussão de Resultados

A implementação da deduplicação introduz um compromisso (*trade-off*) entre a poupança de espaço em disco e o desempenho do sistema. Para analisar a performance dos dois sistemas usamos scripts de testes que fazem recurso das ferramentas anteriormente referenciadas. Para manter o máximo possível de consistência, fizemos recurso de *Direct I/O*, eliminando assim os auxílios dos sistemas operacionais, como por exemplo, a *page cache* que apesar de não termos nenhuma amostra do seu benefício, por vezes duplicava a performance do sistema, influenciando assim os testes.

4.1. Poupança de Espaço

O sistema atinge o seu objetivo principal de poupança de espaço. Em cargas de trabalho com elevada redundância de dados (por exemplo, cópias de ficheiros idênticos ou ficheiros com blocos repetidos), o espaço físico ocupado no ficheiro de *backend* cresce apenas em função dos blocos únicos. A utilização da *freeList* garante que o espaço de ficheiros removidos é efetivamente reaproveitado para novas escritas, evitando a fragmentação externa do armazenamento.

A taxa de deduplicação depende diretamente da percentagem de blocos duplicados na carga de trabalho. Em cenários com 100% de duplicação (por exemplo, escrita repetida do mesmo conteúdo), o sistema armazena apenas uma cópia física, independentemente do número de ficheiros lógicos criados. O tamanho lógico reportado ao utilizador (via `getattr`)

mantém-se correto, refletindo o tamanho total dos dados escritos, enquanto o espaço físico real é significativamente inferior.

A seguir está um print do terminal durante um dos nossos testes, mostrando a efetiva poupança de espaço:

```
>>> RUNNING TEST 1.2: DEDUP_T1.2_D50_J1_write
Executing FIO (write)...
Test 1.2 completed. [100.0%] [w=17.0MiB/s] [w=43
Backend Size: 512M
Logical Size: 1.1G
```

4.2. Desempenho e Sobrecarga

De imediato foi visível nos nossos testes que, como seria de esperar, a simples existência do nosso mecanismo de deduplicação aumentou a latência do sistema nas escritas, uma vez que em vez de apenas guardar o ficheiro também se tem de gerir as nossas estruturas de dados e adicionalmente a computação do SHA512 que também introduz latência. Também ver que o nosso sistema é estável, pois as latências em 95% e 99% do tempo do teste mantiveram-se com valores semelhantes indicando que não houve quedas acentuadas de performance em certos intervalos de tempos.

Leituras:

	Sem fazer deduplicação	Fazer deduplicação sem duplicados
Capacidade de processamento	14.57 MB/s	14.47 MB/s
Latência média	0.267 ms	0.204 ms
Percentil 99 da latência	0.350 ms	0.313 ms
Percentil 95 da latência	0.309 ms	0.362 ms

Escritas:

	Sem fazer deduplicação	Fazer deduplicação sem duplicados
Capacidade de processamento	20.89 MB/s	19.05 MB/s
Latência média	0.186 ms	0.204 ms
Percentil 99 da latência	0.255 ms	0.309 ms
Percentil 95 da latência	0.226 ms	0.251 ms

No entanto, quando a taxa de blocos duplicados cresce ou é bem presente, as vantagens começam a tornar-se óbvias em termos de tempo. Apenas foram feitos testes às escritas, pois estas é que se beneficiam do sistema de deduplicação.

	0% duplicados	50% duplicados	100% duplicados
Capacidade de processamento	19.05 MB/s	28.66 MB/s	52.42 MB/s
Latência média	0.204 ms	0.135 ms	0.073 ms
Percentil 99 da latência	0.309 ms	0.251 ms	0.110 ms
Percentil 95 da latência	0.251 ms	0.222 ms	0.096 ms

Através da ferramenta pidstat para testes com 0% de duplicação, 50% duplicação e 100% duplicação conseguimos obter mais informações, conseguimos observar que quanto maior a duplicação, menor o uso de memória, mas que o uso do processador mantém-se sempre muito semelhante, havendo no entanto uma mudança em como ele é utilizado, pois com pouca duplicação a grande maioria do tempo é usado em Kernel Space (a fazer escritas e leituras), mas quanto maior a taxa de duplicação, mais tempo é passado em User Space a gerir as *hash tables*, para atualizar informações dos blocos.

		0% duplicados	50% duplicados	100% duplicados
Uso de CPU	User Space	5.03%	7.15%	12.64%
	Kernel Space	39.49%	37.14%	28.27%
	CPU total usado	44.52%	44.29%	40.91%
Uso de memória RAM	Memória real (RSS)	27.19MB	17.26MB	7.47MB
	Percentagem de RAM	0.50%	0.32%	0.14%
	Page faults	386.96/s	294.60/s	37.91/s

Em seguida temos também a informação obtida através do *systracer* para casos de teste onde a taxa de duplicação era 0%, 50% e 100% respetivamente. Aqui nós conseguimos ver que quanto maior a deduplicação, menos escritas têm de ser feitas.

Total events processed: 406,851 Capture State: Lost 0 events KERNEL LAYER DISTRIBUTION: syscall : 101,715 events (25.0%) BIO : 101,714 events (25.0%) VFS : 101,711 events (25.0%) EXT4 : 101,711 events (25.0%) TOP OPERATIONS / SYSCALLS: write : 305,120 times pwrite : 101,703 times read : 15 times open : 6 times openat : 4 times INTERCEPTED FILES (Unique Accesses): /proc/sys/crypto/fips_enabled 8192 /backend/DEDUP_T1.1_D0_J1_write.0.0 4096 .sysdata ... and 3 more hidden files.	Total events processed: 272,470 Capture State: Unknown KERNEL LAYER DISTRIBUTION: syscall : 68,987 events (25.3%) VFS : 68,981 events (25.3%) EXT4 : 68,981 events (25.3%) BIO : 65,521 events (24.0%) TOP OPERATIONS / SYSCALLS: write : 206,922 times pwrite : 65,515 times read : 15 times open : 8 times openat : 5 times INTERCEPTED FILES (Unique Accesses): /proc/sys/crypto/fips_enabled .sysdata 4096 DEDUP_T1.2_D50_J1_write.0.0 .metadata ... and 6 more hidden files.	Total events processed: 952 Capture State: Unknown KERNEL LAYER DISTRIBUTION: syscall : 321 events (33.7%) VFS : 315 events (33.1%) EXT4 : 315 events (33.1%) BIO : 1 events (0.1%) TOP OPERATIONS / SYSCALLS: write : 909 times read : 18 times open : 12 times openat : 7 times close : 5 times INTERCEPTED FILES (Unique Accesses): /backend/.metadata DEDUP_T1.3_D100_J1_write.0.0 /etc/ld.so.cache ld.so.cache /usr/lib/ssl/openssl.cnf ... and 8 more hidden files.
END OF SUMMARY: test_write0_dedup	END OF SUMMARY: test_write50_dedup	END OF SUMMARY: test_write100_dedup

Por fim, também fizemos testes ao comportamento do sistema em cenários de escrita *multi-thread*, este teste foi realizado com 4 threads. Com os resultados abaixo apresentados podemos concluir que devido à complexidade da gestão das nossas estruturas de dados aliados à gestão de concorrência das mesmas, é possível verificar que apesar de ter havido um salto de performance no sistema base, a melhoria de performance na nossa implementação foi negligenciável.

	Sem deduplicação single-thread	Sem deduplicação multi-thread	Deduplicação com 0% duplicados single-thread	Deduplicação com 0% duplicados multi-thread
Capacidade de processamento	20.89 MB/s	68.39 MB/s	19.05 MB/s	20.36 MB/s
Latência média	0.186 ms	0.226 ms	0.204 ms	0.763 ms
Percentil 99 da latência	0.255 ms	0.338 ms	0.309 ms	1.270 ms
Percentil 95 da latência	0.226 ms	0.457 ms	0.251 ms	1.794 ms

No entanto, ao analisar a diferença de performance com a adição de deduplicação podemos ver que aqui a performance já registra um salto de ~2x, provavelmente devido a nesta versão não termos de fazer tantas escritas a disco.

	Deduplicação com 0% duplicados single-thread	Deduplicação com 0% duplicados multi-thread	Deduplicação com 50% duplicados single-thread	Deduplicação com 50% duplicados multi-thread
Capacidade de	19.05 MB/s	20.36 MB/s	28.66 MB/s	59.14 MB/s

processamento				
Latência média	0.204 ms	0.763 ms	0.135 ms	0.262 ms
Percentil 99 da latência	0.309 ms	1.270 ms	0.251 ms	0.651 ms
Percentil 95 da latência	0.251 ms	1.794 ms	0.222 ms	0.848 ms

5. Conclusão

O trabalho prático demonstrou a implementação bem-sucedida de um sistema de ficheiros com deduplicação de dados baseado na plataforma FUSE, cumprindo o objetivo de poupar espaço em disco de forma transparente para o utilizador. O sistema alcança este feito através da deduplicação inline de blocos de 4KB, utilizando a assinatura criptográfica SHA-512 para garantir a integridade e evitar colisões. A gestão eficiente do espaço é assegurada pelo mecanismo de contagem de referências e pela freeList para reutilização de blocos após a remoção de ficheiros.

A avaliação experimental, que se baseou na utilização de ferramentas eBPF customizadas, foi crucial para analisar o trade-off entre a poupança de espaço e o desempenho. Embora a introdução do mecanismo de deduplicação e o cálculo do SHA-512 causem um aumento na latência de escrita em cenários de baixa duplicação, os resultados demonstram que o sistema é significativamente mais rápido em cargas de trabalho com elevada redundância. Por exemplo, com 50% de blocos duplicados, a capacidade de processamento aumentou de 19.05 MB/s para 28.66 MB/s em modo single-thread, e registou um salto notável para 59.14 MB/s em modo multi-thread (aproximadamente 2x de ganho). Este aumento de desempenho é acompanhado por uma redução no uso da memória e no tempo gasto em Kernel Space para escritas, transferindo a carga para a gestão de estruturas de dados em User Space.

Em suma, o protótipo valida a premissa da deduplicação como uma estratégia eficaz de otimização de armazenamento.